



Power & Electrical Systems (PES)

Project Autonomous Vehicle (AV)

Telemetry

Technical Manual

University of Puerto Rico, Mayagüez (UPRM)

Completion Date: May 16, 2026

Prepared by: Power and Electrical Systems – Sebastian Pagán Olivera



Table of Contents

1. [Abstract](#)
2. [System Introduction](#)
3. [System Design](#)
4. [Performance & Correct Usage](#)
5. [Appendix](#)



Abstract

This document presents the design and implementation of the telemetry subsystem for B0-R1 within the Project Autonomous Vehicle (AV) team. The manual details the subsystem schematic, electrical distribution, sensing interfaces, embedded control systems, and telemetry data handling processes required for operational monitoring and diagnostics. Topics include sensor integration, voltage and current measurement systems, microcontroller interfacing and user interface implementation. Supporting schematics and operational guidelines are provided to facilitate system integration, maintenance, troubleshooting, and future development. The telemetry subsystem is designed to provide accurate real-time monitoring and reliable system feedback to enhance vehicle safety, performance, and operational efficiency.



System Introduction

Location

The telemetry subsystem is distributed across multiple sections of the autonomous vehicle and is centered around the communication area of the power distribution board (*Figure #2*), where the primary microcontroller is installed. This location (*Figure #6*) serves as the main integration point for subsystem communication, sensor interfacing, and telemetry data processing.

The voltage divider board found in (*Figure #1*) is installed within the battery compartment to support direct power monitoring. The LCD user interface is mounted within the main chassis to provide operators with continuous real-time telemetry information during system operation.

Electrical System Function

The telemetry subsystem is designed to monitor and process electrical and environmental data within the rover's power distribution system (*Figure #2*). The system continuously measures the voltage and current output of the 12 V and 5 V busbars located on the main power distribution board (*Figure #4*) using voltage and current sensors connected to the microcontroller.

Additionally, the subsystem monitors the ambient temperature around the battery to provide thermal awareness of the rover's primary power source during operation. Battery cell voltages from the 4S LiPo battery are measured through a custom voltage divider board as seen in (*Figure #1*), which safely scales the cell tap voltages to levels compatible with the Arduino Mega analog inputs. (*Appendix D*)

All collected sensor data is processed by the Arduino Mega microcontroller and displayed in real time through the LCD user interface mounted on the rover chassis. This telemetry setup provides operational data regarding the rover's power distribution system and battery conditions, supporting safer and more reliable system operation.

Key Objectives

- Monitor 12 V and 5 V busbar's power outputs (*Figure #3*)
- Measure system current consumption
- Monitor ambient temperature inside battery compartment
- Read individual 4S LiPo battery cell voltages through a custom voltage divider board (*Figure #1*)
- Process telemetry data using the Arduino Mega (*Figure #2*)
- Display real-time data on the LCD interface (*Figure #2*)
- Ensure stable sensor communication and signal integrity
- Improve monitoring accuracy of the rover power system
- Provide early warning for overheating components or abnormal conditions
- Support efficient power management and troubleshooting

PROJECT

System Design

Circuit Diagrams & Schematics

Prototype Photos:

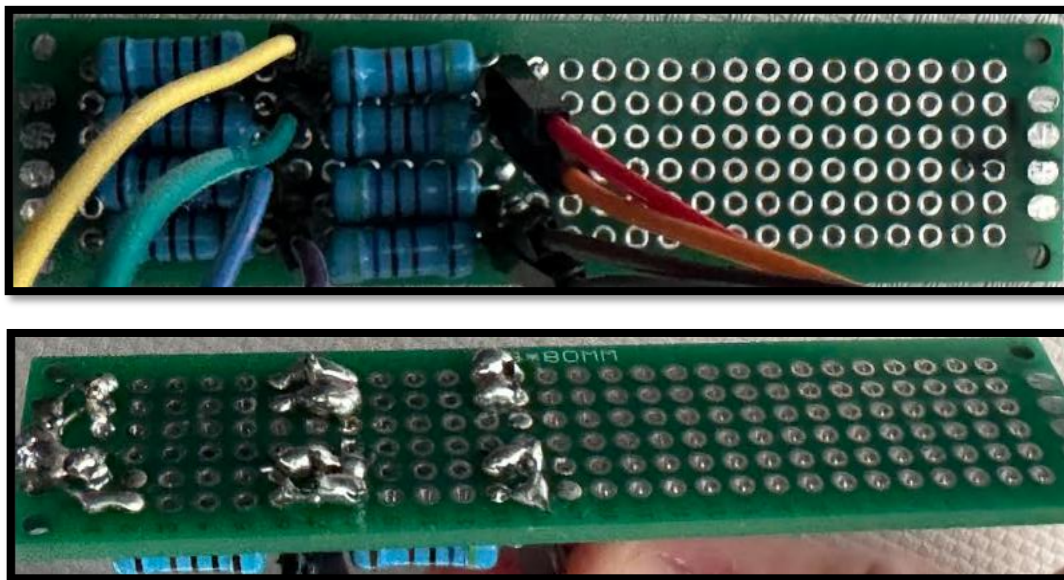


Figure 1. Latest version of Voltage Divider Board

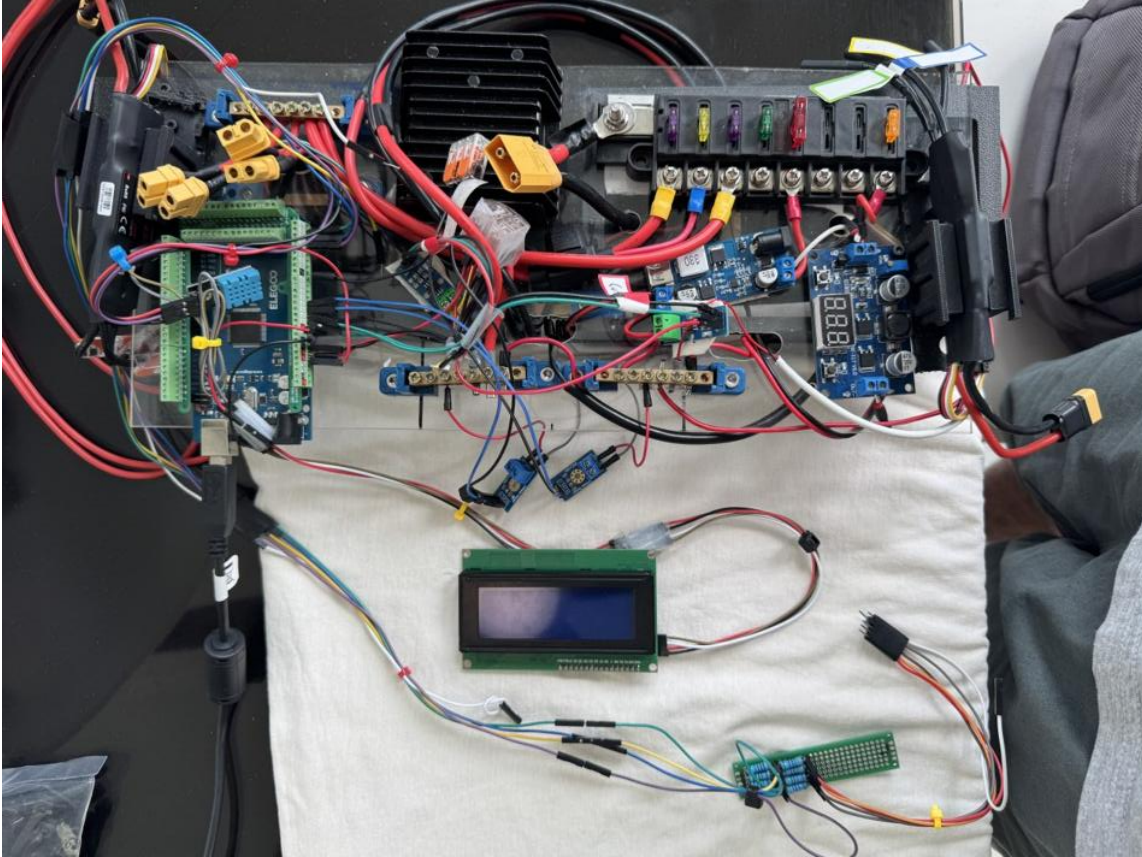


Figure 2. Telemetry System mounted in Power Distribution Board

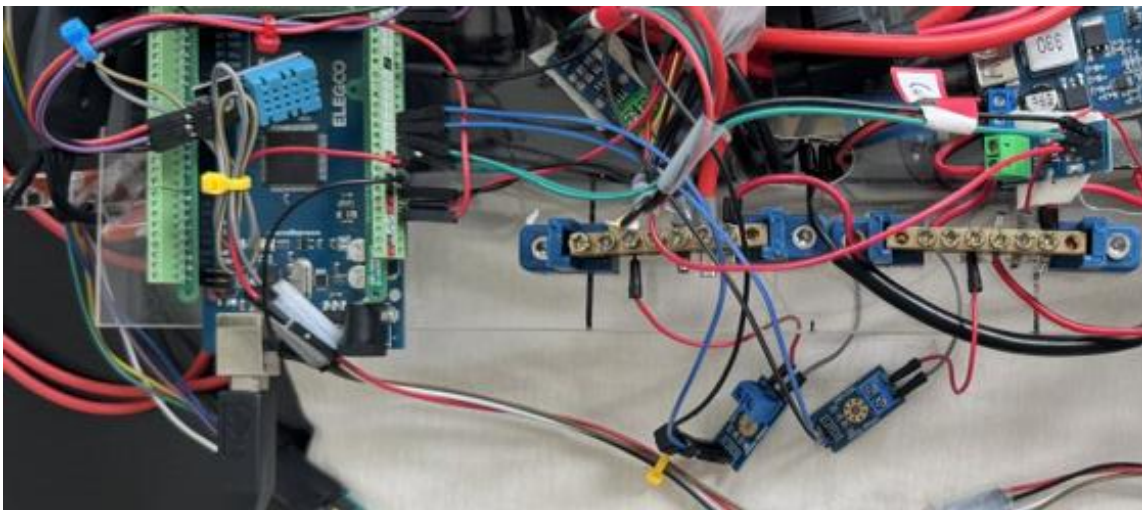


Figure 3. 12V and 5V Breakout Terminals and Sensors Connections

Schematics & Layouts:

NOTES
Busbars are used to represent common nodes between jumper cables

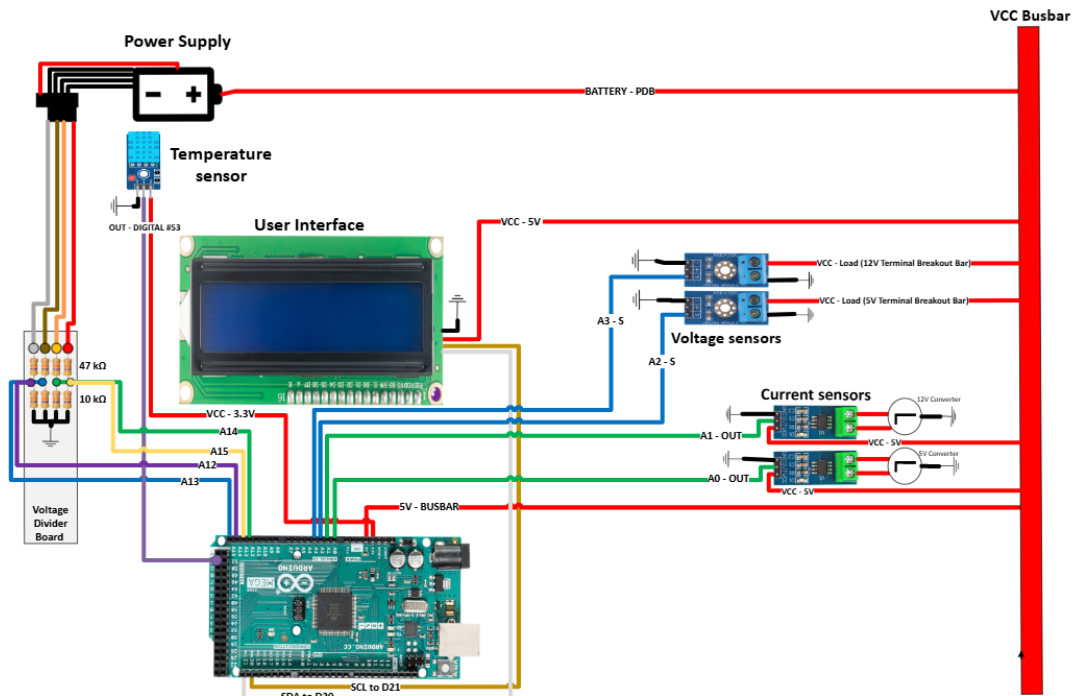


Figure 4. Telemetry System Schematic

[Telemetry Manual Sensor Array Schematic May 11 2026 - Copy.vsd](#)

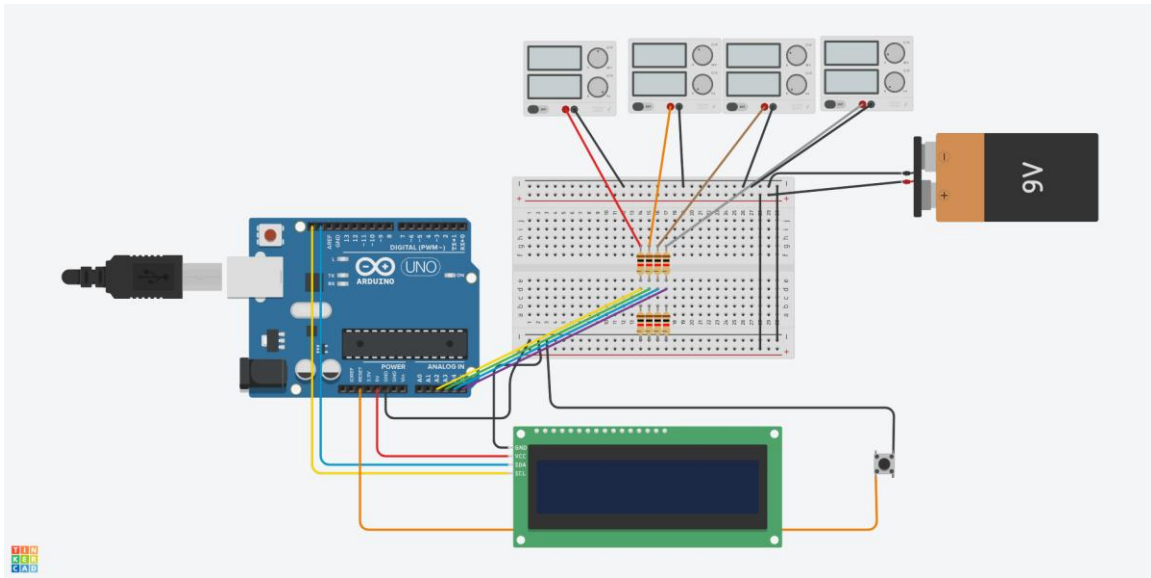


Figure 5. Custom Voltage Divider Board Testing / Simulation in Tinker CAD

https://www.tinkercad.com/things/b8GaJ6fH1cr-sensor-array-system-?sharecode=Qh_ENVkTdBYGfsn-MhKj7oPKEW6_Nu19S8YngfBkXms



Figure 6. Different Sections Inside the Autonomous Rover

Code & Control Logic

Code Repository:

[V3_Sensor_Array.ino](#)

Programmable Logic:

Stage 1 — Startup & Calibration *(Reference Appendix C)*

When the rover is powered, the system runs a one-time setup before entering its monitoring loop.

- The LCD and serial monitor are initialized, so output destinations are ready.
- The DHT11 temperature sensor is initialized on pin 53.
- Current zero calibration is performed automatically. With no load connected, the system takes 200 rapid readings from each current sensor and records the average voltage as the "zero amperes" reference point (I_ZERO). This is necessary because ACS712 current sensors output ~2.5 V at 0 A, not 0 V, so the code must know the baseline before it can calculate real current values. *(Reference Appendix C)*

This is why the startup screen says **"No Load"** --- Connecting loads during calibration will produce permanently incorrect current readings for that session.

Current sensors should be calibrated when the sensor is measuring different loads, *for example: a change in motor drivers for the autonomous vehicle*

Stage 2 — Sensor Reading (*Reference Appendix C*)

Once in the main loop, the system reads three types of sensors on every cycle:

Voltage sensors (A3, A2).

The voltage sensors measure the 12 V and 5 V busbars by reducing the input voltage to a safe range for the Arduino Mega analog pins. The Arduino ADC converts the sensor output into a digital value between 0 and 1023. The firmware then applies to the **V_SCALE** multiplier to convert this ADC value back into the actual measured voltage.

Current sensor (A1, A0).

The ACS712 current sensors measure current flow by producing an output voltage that changes relative to a zero-current reference. When no current is flowing, the sensor outputs a baseline voltage, defined in the code as `I_ZERO`. As current flows through the sensor, the output voltage shifts above or below this reference depending on the current direction. The Arduino reads this signal through analog pins A1 and A0. The firmware subtracts `I_ZERO` from the measured voltage and divides the result by the sensor sensitivity, `I_SENS`, to calculate the current in amperes.

Temperature (pin 53).

The DHT11 temperature sensor is connected to digital pin 53 and is used to monitor the temperature inside the battery compartment. Unlike the voltage and current sensors, DHT11 cannot be read continuously because it requires a delay between measurements. The firmware reads the sensor no faster than every 2.2 seconds to respect this hardware limitation. Before each temperature reading, the code checks the elapsed time from the previous reading. If enough time has passed, the sensor is updated; otherwise, the reading is skipped and the main loop continues running. This prevents DHT11 from delaying the rest of the telemetry system while still providing periodic temperature monitoring.

4S cell taps voltages (A12–A15)

Each analog pin reads a cumulative tap voltage through the 47 k Ω / 10 k Ω voltage divider board. The voltage divider board turns the battery voltage to a safe range for the Arduino (0–5 V). Individual cell voltages are then recovered by subtracting adjacent tap readings:

$$\text{Cell 1} = V1$$

$$\text{Cell 2} = V2 - V1$$

$$\text{Cell 3} = V3 - V2$$

$$\text{Cell 4} = V4 - V3$$

Stage 3 — Data Processing

In this stage, the Arduino Mega converts raw sensor readings into meaningful electrical and environmental values. Voltage readings are converted into volts, current readings into amperes, temperature readings into Celsius degrees, and power is calculated in watts. The firmware continuously tracks the minimum, maximum, and average values for each measurement. This allows the system to monitor performance trends without storing every single reading, reducing memory usage, and keeping the program efficient.

Power is calculated using the relationship:

$$\text{Power (W)} = \text{Voltage (V)} \times \text{Current (A)}$$

Stage 4 — LCD Display (*Reference Appendix B*)

The 20×4 LCD is refreshed every 250 ms with the latest live readings. The temperature shown is the battery compartment reading. If the DHT11 fails to respond, the display shows ERR rather than a stale or misleading value. The LCD connects to the Arduino using just two wires, which keeps the wiring simple and frees up other pins for the rest of the system. When a new reading comes in, the Arduino sends the updated number to a specific spot on the screen. The Arduino communicates with the LCD over I2C, using the SDA and SCL pins listed in the pin assignment table. SDA carries the actual data bits being transferred, while SCL provides the clock pulse that keeps the Arduino synchronized.

On the firmware side, the `LiquidCrystal_I2C` library handles translating sensor values into characters on the display. Each value, voltage, current, temperature, is written to

a fixed row and column using `lcd.setCursor(col, row)` immediately before the corresponding `lcd.print()` call, so the screen layout stays static and only the numbers themselves change as new readings come in. This avoids clearing and redrawing the whole screen every cycle, which would otherwise cause visible flicker.

Two separate serial outputs run on independent timers, so they do not interfere with each other or slow down the main loop:

- Telemetry statistics print every 5 seconds — a full table of min/avg/max for voltage, current, and power on both busbars, plus battery compartment ambient temperature. This is useful for post-run analysis.
- Cell taps voltages print every 500 ms — the four cumulative tap voltages and the four derived cell voltages. This fast update rate is intended for live battery health monitoring.

How the Loop Ties It All Together

After setup and calibration, the telemetry firmware enters the main loop. This loop continuously reads sensor data, processes data, updates the LCD interface, and prints information through the serial monitor. The system uses a non-blocking scheduling structure, meaning each task runs only when its assigned time interval has passed. If a task is not ready, the program skips it and continues with the rest of the loop. This prevents slower components, such as the DHT11 temperature sensor, from delaying voltage readings, current readings, LCD updates, or serial communication. This structure allows the Arduino Mega to manage multiple telemetry functions efficiently on a single processor while maintaining smooth display updates and consistent real-time monitoring.



Performance & Correct Usage

Step-by-Step Protocol:

A. Pre-Power Check

1. Verify that all telemetry wires and sensor connections are secure. (*Figure #2 & #4*) (*Reference Appendix D*)
2. Confirm that the voltage divider board (*Figure #1*) is connected to the 4S LiPo cell taps, as seen in (*Figure #4*)
3. Perform a quick continuity check between 12V and 5V busbars and ground to confirm that no short circuits are present.

B. Startup Check

4. Upload the latest telemetry code to the Arduino Mega.
5. Power on the telemetry system and observe the startup for unstable behavior, incorrect readings or resets. Also look for overheating in components such as sensors, microcontrollers, cables, wagos, busbars, or main battery.
6. Confirm that the LCD displays real-time telemetry values.

C. Sensor Validation

7. Verify current readings using a known load. (current sensors should be calibrated when the sensor is measuring different loads, *for example: a change in motor drivers for the autonomous vehicle*)
8. Confirm that the ambient temperature in the battery compartment is stable and reasonable.
9. Perform a final inspection before rover operation.



Appendix

Appendix A: Bill of Materials (BOM)

The following bill of materials lists the primary components used in the telemetry subsystem. These components are required for voltage monitoring, current measurement, battery cell tap reading, temperature monitoring, microcontroller processing, and user interface display.

Bill of materials

#	Component	Qty	Purpose / Function
1	Arduino Mega 2560	1	Main microcontroller; reads sensor inputs, processes telemetry data, and controls the LCD interface.
2	20×4 I2C LCD Display	1	Displays real-time telemetry values such as voltage, current, temperature, and battery cell data.
3	ACS712 Current Sensor Module	2	Measures current flow from the 12 V and 5 V busbars and outputs an analog voltage signal to the Arduino.
4	0–25 V Voltage Sensor Module	2	Measures the 12 V and 5 V busbar outputs through analog inputs.
5	DHT11 Temperature Sensor	1	Measures the temperature inside the battery compartment.
6	Custom Voltage Divider Board	1	Scales down 4S LiPo cell tap voltages to safe 0–5 V levels for Arduino analog inputs.
7	47 kΩ Resistors	4	Upper resistor values used in the voltage divider circuit.
8	10 kΩ Resistors	4	Lower resistor values used in the voltage divider circuit.
9	4S LiPo Battery Cell Tap Connector	1	Provides access to GND, 1S, 2S, 3S, and 4S battery tap points.
10	Jumper Wires	As needed	Connect sensors, LCD, voltage divider board, and Arduino Mega.
11	Busbar or Terminal Blocks	As needed	Provide secure wiring connections for 12 V and 5 V busbars.
12	Perfboard / Prototype Board	1	Used to build the voltage divider board prototype.
13	USB Cable	1	Used to upload firmware and communicate with the Arduino Mega through the serial monitor.
14	Mounting Hardware	As needed	Secures telemetry components inside the rover chassis.

Appendix B — Sensor and Module Reference

This appendix summarizes the main sensors and modules used in the telemetry subsystem.

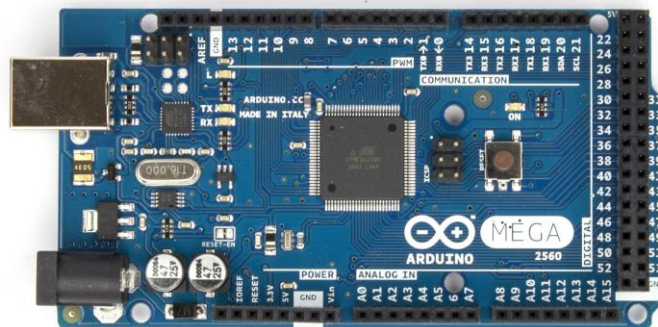
Arduino Mega 2560

The Arduino Mega 2560 serves as the main processing unit for the telemetry subsystem. It reads analog signals from the voltage sensors, current sensors, and voltage divider board. It also reads the DHT11 temperature sensor through a digital pin and sends processed telemetry values to the LCD screen and serial monitor.

Main functions:

- Reads voltage sensors through analog pins
- Reads ACS712 current sensors through analog pins
- Reads 4S LiPo cell tap voltages through the voltage divider board
- Reads ambient temperature in the battery compartment through the DHT11 sensor
- Processes raw sensor values into useful information
- Displays telemetry data on the 20×4 LCD
- Prints real-time data through the serial monitor

Component image:



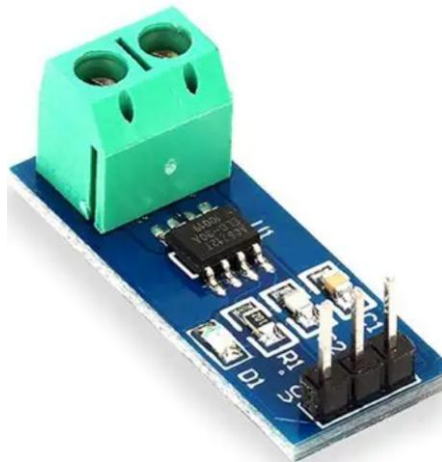
ACS712 Current Sensor

The ACS712 current sensor measures current flow and outputs a voltage signal proportional to the measured current. This means that the sensors don't report current (amps) directly; it reports a voltage that goes up or down in a predictable way as the current changes. So, if the current increases, the voltage coming out of the sensor increases too, in a straight-line (proportional) relationship. The Arduino reads that voltage on one of its analog pins, and then the firmware does the math to convert voltage back into amps of current, using a known formula for that sensor. When no current is flowing, the sensor output is approximately half of the supply voltage. This value is used as the zero-current reference, defined in the firmware as `I_ZERO`.

Important notes:

- Requires zero-current calibration before use
- Output changes above or below the zero-current reference depending on current direction
- Current is calculated using the sensor sensitivity value, `I_SENS`
- Incorrect calibration can cause inaccurate current readings

Component image:



Voltage Sensor Module

The voltage sensor modules are used to monitor the 12 V and 5 V busbars. These modules reduce the input voltage to a safe analog voltage level that can be read by the Arduino Mega.

Important notes:

- Arduino analog inputs must remain within 0–5 V
- Voltage readings should be verified with a multimeter
- The voltage scale factor, V_SCALE , is used in the firmware to convert ADC readings into real voltage values

Component image:



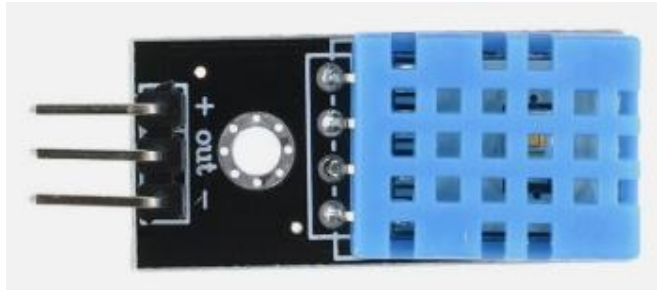
DHT11 Temperature Sensor

The DHT11 temperature sensor is used to monitor the battery compartment temperature. This provides basic thermal awareness of the rover's power source area.

Important notes:

- Connected to digital pin 53
- Should not be read continuously
- Firmware reads the sensor no faster than approximately every 2.2 seconds
- If the sensor fails to respond, the LCD displays ERR instead of an outdated value

Component image:



Appendix C — Calibration Procedures

This appendix provides calibration steps for improving telemetry accuracy.

Current Sensor Calibration

The ACS712 current sensors require zero-current calibration before accurate readings can be calculated. During startup, the system should be powered with no load connected. The firmware takes multiple readings from each current sensor and calculates the average output voltage. This value becomes the zero-current reference, I_ZERO .

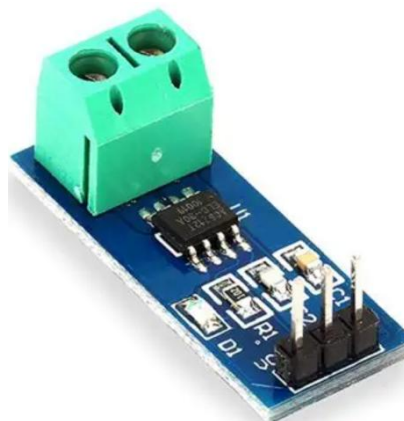
Procedure:

1. Disconnect all loads from the monitored current line.
2. Power on the telemetry subsystem.
3. Allow the firmware to complete startup calibration.
4. Do not connect loads while the startup screen displays the no-load calibration message.
5. After calibration, connect the intended load and verify current readings.

Important:

Connecting a load during calibration will cause incorrect current readings for that session.

Component image:



Voltage Sensor Calibration

Voltage sensors should be calibrated by comparing the Arduino reading with a multimeter measurement from the same monitored terminal.

Procedure:

1. Measure the actual voltage using a multimeter.
2. Record the voltage displayed by the telemetry system.
3. Calculate the correction factor:

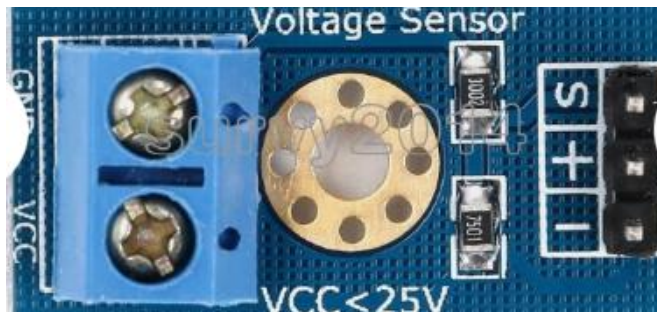
Correction Factor = Multimeter Voltage ÷ Arduino Displayed Voltage

4. Update the voltage scale factor in the firmware:

New V_SCALE = Current V_SCALE × Correction Factor

5. Upload the updated firmware and verify the corrected reading.

Component image:



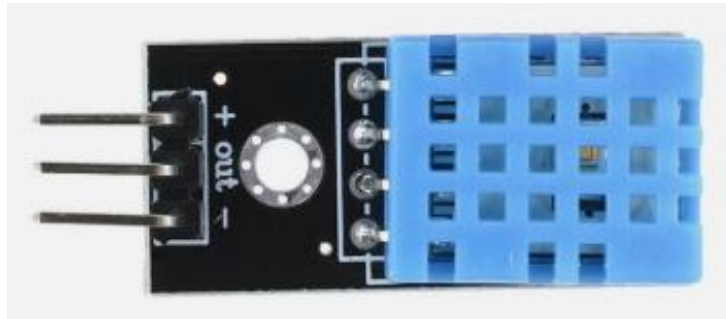
Temperature Sensor Check

The DHT11 temperature sensor does not require the same type of calibration as the current and voltage sensors. However, its reading should be checked for reasonableness.

Procedure:

1. Confirm that the DHT11 is installed inside the battery compartment.
2. Power on the telemetry subsystem.
3. Wait for the sensor reading to update.
4. Compare the displayed temperature with the approximate ambient temperature.
5. If the LCD displays ERR, inspect the sensor wiring and firmware pin assignment.

Component image:



Appendix D — Pin Assignment Reference

The following table summarizes the main Arduino Mega pin assignments used by the telemetry subsystem.

Arduino Mega Pin Assignments

Pin	Connected Component	Pin Type	Purpose
A0	ACS712 Current Sensor 1	Analog	Measures current on 5V busbar.
A1	ACS712 Current Sensor 2	Analog	Measures current on 12V busbar.
A2	Voltage Sensor 1	Analog	Measures voltage on 5V busbar.
A3	Voltage Sensor 2	Analog	Measures voltage on 12V busbar.
A12	4S LiPo Tap 1 through voltage divider	Analog	Reads cumulative battery tap voltage V1.
A13	4S LiPo Tap 2 through voltage divider	Analog	Reads cumulative battery tap voltage V2.
A14	4S LiPo Tap 3 through voltage divider	Analog	Reads cumulative battery tap voltage V3.
A15	4S LiPo Tap 4 through voltage divider	Analog	Reads cumulative battery tap voltage V4.
Pin 53	DHT11 Temperature Sensor	Digital	Reads battery compartment temperature.
SDA	20×4 I2C LCD	I2C Data	Sends display data to LCD.
SCL	20×4 I2C LCD	I2C Clock	Provides an I2C clock signal for LCD communication.
5 V	Sensors / LCD	Power	Provides regulated 5 V supply to low-voltage components.
GND	Common Ground	Reference	Common reference point.

Note: All sensor grounds must be connected to the Arduino Mega ground to ensure accurate analog readings.

Appendix E — Glossary of Terms

Term	Definition
ADC	Analog-to-Digital Converter. Converts an analog voltage signal into a digital value that the Arduino can process.
ACS712	Current sensor module used to measure electrical current.
Cell Tap	A connection point on the battery pack used to measure individual cell voltages.
Current	The flow of electrical charge, measured in amperes.
DHT11	Digital temperature and humidity sensor are used in this system for battery compartment temperature monitoring.
I2C	Communication protocol used by the LCD display to communicate with the Arduino.
I_ZERO	Zero-current reference value used to calibrate the ACS712 current sensors.
LiPo	Lithium Polymer battery, used as the rover's power source.
Non-Blocking Scheduling	A programming method where tasks run only when due instead of stopping the entire loop.
Power	Electrical energy usage rate, calculated as voltage multiplied by current.
Telemetry	The process of collecting, processing, and displaying system data for monitoring and decision-making.
Voltage Divider	A resistor circuit used to reduce a higher voltage to a lower measurable voltage.
V_SCALE	Firmware multiplier used to convert voltage sensor readings into actual voltage values.

Appendix F — Safety and Handling Notes

The telemetry subsystem connects to sensitive electrical components and the main rover battery. Proper handling is required to avoid inaccurate readings, short circuits, or hardware damage.

Safety guidelines:

- Disconnect the battery before modifying the wiring.
- Never connect battery cell taps directly to Arduino analog pins without the voltage divider board.
- Verify polarity before connecting voltage sensors.
- Keep analog signal wires away from high-current power wires when possible.
- Confirm that all grounds are common between the Arduino, sensors, and monitored systems.
- Use a multimeter to verify voltage levels before connecting to the Arduino.
- Inspect the voltage divider board for solder bridges or loose connections before operation.
- Do not power the system if any component overheats or if the Arduino resets repeatedly.